Python Topic 8 — Conditionals (if / elif / else)

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert

🌱 Layer 1 — New to Python

Conditionals let your code make decisions — "if this is true, do that."

```
age = 20

if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

Three things that look different from JavaScript right away:

- **No parentheses** around the condition (though allowed)
- **A colon** `:` at the end of the line
- **Indentation** (spaces) instead of `{ }` braces to group the block

```
// JS — braces and parentheses
if (age >= 18) {
    console.log("adult");
}

# Python — colon and indentation
if age >= 18:
    print("adult")
```

👩‍💻 Layer 2 — Intermediate (real code + how it works)

`elif` = "else if"

```
score = 75

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
```

```
    grade = "C"
else:
    grade = "F"
```

Python checks each condition top to bottom and runs the **first** one that's true, then skips the rest. `elif` is one word (not `else if` like JS).

Indentation IS the syntax

The biggest adjustment from JS. The 4-space indent isn't just style — it *defines* the block.

```
if age >= 18:
    print("adult")       # inside the if (indented)
    print("can vote")    # also inside the if
print("done")            # OUTSIDE the if (not indented) — always runs
```

⚠️ **Note:** In JS, indentation is cosmetic and `{ }` define blocks. In Python, **indentation is mandatory and meaningful**. Mixing tabs and spaces causes errors — pick 4 spaces and stay consistent (your editor handles this).

The ternary (conditional expression)

```
# JS: const status = age >= 18 ? "adult" : "minor";
status = "adult" if age >= 18 else "minor"
```

ℹ️ **Word order:** Python reads like English — *value* `if` *condition* `else` *value*. Different from JS's `condition ? a : b`.

Combining conditions (recall Topic 4)

```
if age >= 18 and has_id:
    print("entry allowed")

if user_role == "admin" or user_role == "moderator":
    print("can edit")
```

🧠 Layer 3 — Expert (internals & gotchas)

1. Truthiness in conditions (recall Topic 4)

```
items = []

# ❌ verbose
```

```python
if len(items) > 0:
    ...

# ✅ Pythonic — empty list is falsy
if items:
    print("has items")

if not items:
    print("empty")
```

Same for strings, dicts, `None`. Writing `if x == True:` is un-Pythonic — just write `if x:`.

2. `match` statement — Python's `switch` (3.10+)

```python
command = "start"

match command:
    case "start":
        print("Starting...")
    case "stop":
        print("Stopping...")
    case "pause" | "wait":      # multiple values with |
        print("Pausing...")
    case _:                     # _ is the default (like default:)
        print("Unknown command")
```

But `match` is far more powerful than JS `switch` — it does **structural pattern matching**, destructuring data:

```python
point = (0, 5)

match point:
    case (0, 0):
        print("origin")
    case (0, y):              # matches any (0, something), binds y
        print(f"on Y axis at {y}")
    case (x, 0):
        print(f"on X axis at {x}")
    case (x, y):
        print(f"at {x}, {y}")
```

This destructures dicts, classes, and nested structures — closer to Rust/Scala pattern matching than a plain switch.

3. No fall-through

Unlike JS `switch`, `match` cases don't fall through — no `break` needed. Each case is isolated.

4. Guard clauses — avoid deep nesting

```
# ❌ deeply nested (arrow code)
def process(user):
    if user is not None:
        if user.is_active:
            if user.has_permission:
                return "success"

# ✅ guard clauses — return early, stay flat
def process(user):
    if user is None:
        return "no user"
    if not user.is_active:
        return "inactive"
    if not user.has_permission:
        return "no permission"
    return "success"
```

✅ **Habit:** guard clauses (early returns) keep code readable and flat — interviewers love it.

5. The walrus operator `:=` — assign AND test (3.8+)

```
# Instead of:
data = get_data()
if data:
    process(data)

# One line — assign to `data` AND check it:
if data := get_data():
    process(data)
```

Handy in loops and conditions to avoid recomputing. Use sparingly for readability.

🎯 Interview Q&A Cards

Q: How do Python conditionals differ from JS syntactically?

A: Colon `:` + indentation instead of `{ }`, no required parentheses, and `elif` as one keyword. Indentation is meaningful, not cosmetic.

Q: Write the ternary word order.

A: `value_if_true if condition else value_if_false` — e.g. `"adult" if age >= 18 else "minor"`. The value comes first, unlike JS's `cond ? a : b`.

Q: How is `match` different from JS `switch`?

A: `match` does structural pattern matching (destructures tuples, dicts, classes), has no fall-through (no `break`), and uses `case _` as default. It's far more than value comparison.

Q: What's a guard clause?

A: An early return that handles an edge/invalid case up front, keeping the main logic flat instead of deeply nested. Improves readability.

🪝 **Memory hook:** *Colon + indent, not braces. Ternary reads like English: `a if cond else b`. `match` destructures, `switch` just compares.*

✅ **One-liner for interviews:** "Python conditionals use `if` / `elif` / `else` with colons and indentation instead of braces. The ternary reads `a if cond else b`. `match` (3.10+) does structural pattern matching, not just value switching, with no fall-through. Prefer truthiness checks and guard clauses over deep nesting."